

NovaLCT, devo collegare la centralina prima di avviarlo o posso farlo anche dopo?

2 hours ago

Puoi collegare la centralina (Sending Card) **sia prima che dopo** l'avvio di NovaLCT, ma la procedura consigliata prevede il collegamento fisico prima dell'apertura del software per un riconoscimento automatico immediato.  YouT... +1

Se colleghi prima di avviare

- Al lancio, NovaLCT scansiona le porte USB/COM e rileva automaticamente l'hardware collegato.
- Nella finestra principale vedrai subito il numero di "Control System" rilevati (es. "1").  NovaS... +1

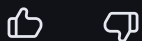
Se colleghi dopo aver avviato

- Il software potrebbe non mostrare subito il dispositivo. In questo caso, devi forzare il rilevamento cliccando sul tasto **Refresh** (Aggiorna) nella schermata principale.
- In alternativa, puoi andare nel menu **System** e selezionare **Reconnect** per ripristinare la comunicazione senza riavviare il programma.  YouTube +2

Nota importante: Assicurati che i driver USB siano installati correttamente; spesso vengono configurati automaticamente durante l'installazione di NovaLCT. Una volta stabilita la connessione, dovrai effettuare il login come **Advanced Synchronous System User** (password predefinita: `admin`) per accedere alla configurazione dello schermo.  YouTube +3

Hai già installato i **driver USB** correttamente o il software non vede ancora la centralina nonostante il refresh?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



  8 siti

How Do You Configure An LED Wall? feat. Novastar Processors
To configure an LED wall, connect the video input from the PC to the HDMI port of the NovaStar processor. If a third-party video s...

 YouTube · Above AVL Gear



NovaLCT LED Configuration Tool for Multimedia Player User Manual

XI'AN NOVASTAR TECH CO.,LTD. ... Enter the user name and password, and then click OK. For example, the login user name and the def...

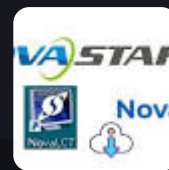
 NovaStar



NovaStar NovaLCT Download, Setup & Configuration Guide ...

Run the installer with administrator privileges. Install the necessary USB or network drivers. Restart the computer if prompted. L...

 UNIT LED



Mostra tutto

mi puo spiegare come fa il software a riconoscere la centralina? invia dei pacchetti specifici? è possibile recuperar... 

2 hours ago

Il riconoscimento della centralina da parte di NovaLCT avviene attraverso un processo di scansione delle porte di comunicazione (USB o Ethernet) e l'invio di pacchetti di interrogazione specifici basati su un protocollo proprietario.  YouT... +1

Come avviene il riconoscimento

- **Scansione Porte:** All'avvio o premendo "Refresh", il software interroga tutte le porte seriali (COM) create dai driver USB o le interfacce di rete attive.
- **Pacchetti di Handshake:** NovaLCT invia pacchetti strutturati con un'intestazione specifica (**Packet Header**), il contenuto del comando (**Protocol Content**) e un codice di controllo (**Checksum**).
- **Identificazione Hardware:** La centralina risponde inviando i propri dati identificativi (modello, versione firmware, numero di porte). Se la risposta è corretta, il software visualizza il dispositivo nella schermata principale.  Daktronics +4

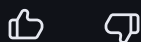
Manuali e Protocolli

Novastar non distribuisce pubblicamente un "manuale del protocollo" completo per gli utenti finali, poiché si tratta di tecnologia proprietaria. Tuttavia, esistono diverse risorse per approfondire:

1. **Protocollo COM (M3):** Esistono documenti tecnici come il [Nova M3 Control System COM Protocol](#) che descrivono i pacchetti per comandi base come luminosità, temperatura e versione firmware.
2. **Protocollo Central Control:** Per l'integrazione con sistemi domotici (es. Crestron), è disponibile il [NovaStar Central Control Protocol](#) che utilizza TCP (porta 5200) o UDP (porta 5201) per il controllo remoto.
3. **Reverse Engineering & SDK:** Per gli sviluppatori, esistono librerie non ufficiali su [GitHub \(come novastar-codec\)](#) che hanno decodificato parte del protocollo seriale e di

Ti interessa integrare il controllo della centralina in un **software personalizzato** o hai solo bisogno di risolvere un **problema di connessione** con NovaLCT?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

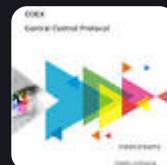


 9 siti

NovaStar COEX Central Control Protocol - Manuals.plus

2.2 TCP Communication Protocol. The PC and controller are connected via Ethernet. The PC sends TCP packets to port 5200 on the con...

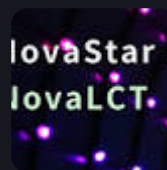
 Manuals+ 



NovaStar NovaLCT Software: A Complete Beginner's Guide

In synchronous mode, the PC is only used to control the LED display wall, and the media source is transferred by DP/HDMI/DVI cable...

 LedInCloud 



Instructions Instructions - NovaStar

... . 1. RS232 Serial Communication Protocol

 NovaStar 



Mostra tutto

Mi interessa integrare il controllo della centralina in un software personalizzato

2 hours ago

Integrare il controllo delle centraline NovaStar in un software personalizzato è possibile, ma la strada da seguire dipende dal modello di hardware che stai utilizzando (serie classica o serie nuova COEX). 

Ecco le opzioni principali per uno sviluppatore:

1. Per Centraline Classiche (MCTRL, VX, MSD)

NovaStar non pubblica un SDK ufficiale aperto per queste serie (è spesso sotto NDA), ma la community ha effettuato il reverse engineering del protocollo:

- **Librerie Open Source:** Esiste un eccellente wrapper API chiamato [novastar-codec](#) ([GitHub](#)), scritto in TypeScript/JavaScript. Questo progetto include pacchetti per la

comunicazione via [Seriale/USB](#) e [Rete/TCP](#).

- **Sniffing del Protocollo:** Se preferisci scrivere il tuo codice in C# o C++, puoi usare il Dissettore Wireshark per NovaStar per analizzare i pacchetti inviati da NovaLCT e replicarli nel tuo software.
- **Protocollo RS232:** Alcuni modelli supportano comandi base (luminosità, switch ingressi) tramite protocollo seriale documentato in manuali tecnici come il NovaStar RS232 Protocol.  GitHub +2

2. Per Serie Nuova (COEX / VMP)

Se utilizzi i nuovi processori della serie COEX, NovaStar è molto più aperta:

- **API HTTP nativa:** Questi dispositivi supportano chiamate API via HTTP (porta 8001).
- **Documentazione ufficiale:** Puoi consultare la [Guida API COEX](#) che spiega come inviare richieste JSON per regolare parametri dello schermo o cambiare preset. 

3. Per Player Multimediali (Serie Taurus / TB)

Per i player che gestiscono contenuti (come il TB60), puoi utilizzare:

- **NovaCloud / VNNOX API:** Per il controllo remoto via cloud.
- **Terminal Open API:** Una documentazione specifica per l'integrazione di sistemi di terze parti con i player.  屏... +1

Consiglio tecnico: Se stai sviluppando su Windows in .NET, puoi provare a de-compilare le DLL presenti nella cartella di installazione di NovaLCT

(es. `Nova.SmartLCT.Database.dll` o simili) per vedere come il software ufficiale invia i comandi, dato che molte funzioni sono mappate internamente come metodi C#.  GitH... +1

Quale **linguaggio di programmazione** intendi usare e su quale **modello di centralina** specifico vuoi lavorare?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



7 siti



API wrapper for working with NovaStar devices - GitHub

js. This JavaScript library (written in TypeScript) is an attempt, using reverse engineering, to write an API wrapper for working ...



GitHub



novastar/packages/native/README.md at main - GitHub

Important note This repository contains C# scripts, some wrapping for the generated API, and the original binary libraries. All us...

COEX API Guidance

Introduction. The API request type for COEX is HTTP. In online mode, the controller IP is the actual device IP, with port number 8...

 COEX Wiki 



Mostra tutto

puoi usare il Dissettoire Wireshark per NovaStar?

2 hours ago

Sì, puoi utilizzare un **dissettoire Wireshark** specifico per analizzare il traffico delle centraline NovaStar. Poiché il protocollo è proprietario e binario, vederlo in formato "grezzo" (esadecimale) sarebbe molto difficile da interpretare senza questo strumento.  GitHub +2

Come installarlo e usarlo

Il progetto open source più affidabile per questo scopo è il [dissettoire Lua di sarakusha](#) (GitHub). 

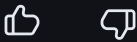
1. **Scarica i file:** Ti servono i file `novastar.lua` e `addressMapping.lua` dalla cartella `wireshark` del repository.
2. **Copia i file:** Inseriscili nella cartella dei plugin personali di Wireshark:
 - **Windows:** `%APPDATA%\Wireshark\plugins`
 - **Linux/macOS:** `~/.local/lib/wireshark/plugins`
3. **Attiva il plugin:** Apri Wireshark (o premi `Ctrl+Shift+L` se è già aperto) per ricaricare i plugin Lua.
4. **Filtra il traffico:** Digita `novastar` nella barra dei filtri di ricerca. Wireshark inizierà a tradurre i pacchetti TCP/UDP mostrando i nomi dei comandi (es. *Set Brightness*, *Read Firmware Version*) e i relativi parametri in chiaro.  GitHub +3

Consigli per il Reverse Engineering

- **Isola il traffico:** Avvia la cattura sull'interfaccia di rete collegata alla centralina e poi esegui un'azione singola su NovaLCT (es. cambia la luminosità dal 10% al 20%). Il dissettoire ti mostrerà esattamente quale pacchetto è stato inviato per quell'operazione.
- **Disabilita il riassetto:** Nelle preferenze di Wireshark, è consigliabile impostare il profilo su "classic" o disabilitare il "TCP reassembly" per evitare conflitti con la decodifica del protocollo NovaStar.  GitHub +2

Vuoi procedere con l'analisi di una **connessione USB** (tramite porta COM virtuale) o stai monitorando una centralina collegata via **Ethernet**? 

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



  8 siti

API wrapper for working with NovaStar devices - GitHub
Wireshark Lua/Dissector. Using Dissector (`./wireshark/novastar.lua`) you can independently examine the novastar exchange protoco...

 GitHub 



cedric-uden/Novastar-Controller: HowTo and Calls to ... - GitHub
This repo stores scripts used to control NovaStar controllers without using the clumsy and error-prone interface. The scripts can ...

 GitHub 



How to Sniff Packets using Wireshark
Wireshark is a tool used to inspect data passing through a network interface, commonly Wi-Fi but also Ethernet. Network transmissi...

 YouTube · Mental Outlaw 



Mostra tutto

Per ora sto monitorando una centralina collegata via Ethernet ma sono interessato anche alla connessione via USB. Per ... 

2 hours ago

Capisco perfettamente la situazione: fare reverse engineering senza l'hardware fisico è come cercare di completare un puzzle al buio. Purtroppo, essendo protocolli industriali, non esiste un repository pubblico ufficiale di "catture" (file `.pcap`), ma posso darti dei riferimenti e dei dati tecnici che ti faranno risparmiare ore di sniffing.

1. Dove trovare dati già sniffati

La risorsa più preziosa per te al momento è il codice sorgente di chi ha già fatto questo lavoro. Invece di guardare un file di cattura statico, guarda come sono strutturati i pacchetti nei parser già scritti:

- **Repository `novastar-codec` (TypeScript):** Guarda il file `protocol.ts`. Qui troverai la definizione della struttura dei pacchetti (Header, Device Type, Address, Length, Checksum).

- **Mappe degli Indirizzi:** Il cuore del protocollo NovaStar è basato su registri. Ad esempio, la luminosità è quasi sempre all'indirizzo `0x02000002`. Molti di questi indirizzi sono mappati nel file `addressMapping.lua` del dissettore che ti menzionavo prima.

2. Struttura del Pacchetto (Protocollo M3)

Le serie MCTRL660 e PRO usano generalmente il protocollo seriale/UDP basato su pacchetti di circa 10-15 byte per i comandi semplici. Ecco una struttura tipo che troveresti nello sniffing:

Byte	Nome	Descrizione
0-1	Header	Solitamente <code>0x55 AA</code> (Richiesta) o <code>0xAA 55</code> (Risposta)
2	Device Type	<code>0x00</code> per la Sending Card
3	Device Index	<code>0x00</code> se hai una sola centralina
4	Command Type	<code>0x01</code> Read, <code>0x00</code> Write
5-8	Register Address	L'indirizzo di memoria su cui operare
9-10	Data Length	Quanti byte leggere/scrivere
11...	Data	Il payload effettivo
Last	Checksum	Somma dei byte precedenti (spesso XOR o somma semplice)

3. Differenze tra USB (COM) ed Ethernet

- **Ethernet (UDP/TCP):** I pacchetti sopra descritti sono semplicemente incapsulati dentro un frame UDP (solitamente porta **5200**). NovaLCT invia un pacchetto di "Broadcast" per trovare le centraline sulla rete locale.
- **USB:** La centralina viene vista come una porta **COM virtuale** (driver FTDI o Silicon Labs). Il software apre la porta seriale e invia esattamente la stessa sequenza di byte che invierebbe via UDP. Se vuoi sniffare l'USB senza hardware, puoi usare un software come **com0com** per creare porte virtuali accoppiate e vedere cosa scrive NovaLCT.

4. Il protocollo "PRO" (MCTRL660 PRO)

Attenzione: la versione **PRO** supporta anche il controllo via interfaccia web. Questo significa che spesso comunica tramite **JSON su HTTP** (Porta 80) o protocolli più moderni rispetto alla 660 standard.

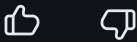
Cosa posso fare per aiutarti:

Se hai un comando specifico che stai cercando di decodificare (es. "cambio ingresso" o "set

contrasto"), posso provare a darti l'indirizzo del registro corrispondente o la stringa esadecimale probabile.

Ti servirebbe che ti fornissi un esempio di **pacchetto esadecimale tipico** per la lettura della luminosità o preferisci approfondire la gestione della porta COM virtuale?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



per ora mi sto concentrando sulla connessione ethernet e si, mi interesserebbe sapere la stringa esadecimale con cui la ...



2 hours ago

Il processo di "scoperta" (Discovery) via Ethernet è il punto di partenza fondamentale. Per le centraline come la **MCTRL660 PRO**, NovaLCT non aspetta passivamente, ma invia dei pacchetti **UDP Broadcast** per sollecitare una risposta.

Ecco come avviene lo scambio esadecimale nel protocollo NovaStar (serie M3/M4):

1. Il pacchetto di "Richiesta Presenza" (NovaLCT -> Centralina)

NovaLCT invia solitamente un pacchetto sulla porta **UDP 5200** all'indirizzo di broadcast (es. `255.255.255.255` o il broadcast della sottorete).

La stringa tipica di "interrogazione" è strutturata così:

```
55 AA 00 00 01 00 00 00 00 00 01 00 57
```

- **55 AA**: Header di comando (Request).
- **00 00**: Device Type e Index (Broadcasting a tutti).
- **01**: Tipo comando (Read).
- **00 00 00 00**: Indirizzo di registro (0x00 è il registro ID/Version).
- **01 00**: Lunghezza dati richiesta (1 byte).
- **57**: Checksum (calcolato sulla somma dei byte).

2. La risposta della centralina (MCTRL660 PRO -> NovaLCT)

La centralina risponde (Unicast) all'IP del computer. La risposta contiene i dati identificativi. Una risposta "tipo" per una 660 PRO è molto più lunga perché include il nome del modello e la versione firmware.

Esempio di struttura della risposta:

```
AA 55 00 00 00 00 00 00 00 00 20 00 [DATI_HARDWARE] [CHECKSUM]
```


- **AA 55**: Header di risposta (Response).
- **20 00**: Indica che seguono 32 byte di dati descrittivi.
- **[DATI_HARDWARE]**: Qui troverai una stringa ASCII o dei codici identificativi. Per la MCTRL660 PRO, vedresti nel payload esadecimale riferimenti come `4D 43 54 52 4C 36 36 30 50 72 6F` (che tradotto in ASCII è proprio **MCTRL660Pro**).

3. Come simulare la risposta senza hardware

Se stai scrivendo un emulatore per testare il tuo software:

1. Mettiti in ascolto sulla porta **UDP 5200**.
2. Quando ricevi il pacchetto che inizia con `55 AA`, rispondi con un pacchetto che inizia con `AA 55`.
3. Nel payload (dopo l'header e l'indirizzo), inserisci la stringa identificativa del modello che vuoi emulare.

Risorsa per Sniffing già pronti

Dato che non hai l'hardware, la risorsa "oro" per te sono i **test case** dei progetti open source. Nel repository `novastar-codec / packages / serial`, gli sviluppatori caricano spesso file di log o definizioni di pacchetti estratti da sessioni reali.

In particolare, controlla i file `.spec.ts` o i mock: contengono le sequenze di byte esatte usate per il login e il riconoscimento.

Prossimo passo consigliato: Vuoi che ti aiuti a decifrare il calcolo del **Checksum** (fondamentale per far sì che NovaLCT accetti la tua risposta simulata) o ti serve la mappa degli indirizzi per la **luminosità**?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



si mi servirebbe il calcolo del Checksum

2 hours ago

Il calcolo del **Checksum** nel protocollo NovaStar è fortunatamente molto semplice: **si tratta di una somma aritmetica a 16 bit (Addition Checksum) di tutti i byte del pacchetto, esclusi l'header e il checksum stesso**.

Ecco la logica esatta per i pacchetti delle centraline serie M3 (come la MCTRL660):

1. La Formula

Dato un pacchetto standard:

[Header (2 byte)] [Dati del Comando (N byte)] [Checksum (2 byte)]

1. **Ignora** i primi due byte dell'header (`55 AA` o `AA 55`).
2. **Somma** tutti i byte successivi uno per uno.
3. **Aggiungi** al totale il valore fisso `0x5555` (questo è l'offset "magico" del protocollo NovaStar).
4. Il risultato finale deve essere troncato a **16 bit** (2 byte).
5. Il checksum viene solitamente inviato in formato **Little Endian** (byte meno significativo per primo).

2. Esempio Pratico

Immaginiamo di voler calcolare il checksum per un pacchetto di lettura della luminosità (Read, Register `0x02000002` , Length 1):

Pacchetto (senza checksum): `55 AA 00 00 01 02 00 00 02 01 00`

- **Header:** `55 AA` (lo saltiamo)
- **Payload:** `00, 00, 01, 02, 00, 00, 02, 01, 00`
- **Somma dei byte:** $0x00 + 0x00 + 0x01 + 0x02 + 0x00 + 0x00 + 0x02 + 0x01 + 0x00 = 0x06$
- **Aggiungi Offset:** $0x06 + 0x5555 = 0x555B$
- **Risultato:** Il Checksum è `5B 55` (in Little Endian).

3. Implementazione in C# (Esempio)

Se stai sviluppando in C# o C++, ecco una funzione rapida:

csharp

```
public byte[] CalculateChecksum(byte[] payload) {
    int sum = 0x5555;
    foreach (byte b in payload) {
        sum += b;
    }
    // Troncamento a 16 bit e conversione in array di 2 byte
    return new byte[] { (byte)(sum & 0xFF), (byte)((sum >> 8) & 0xFF) };
}
```



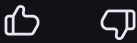
4. Particolarità delle serie PRO

Mentre la MCTRL660 standard usa quasi sempre questo metodo a 16 bit, la **MCTRL660 PRO** quando comunica via rete può utilizzare pacchetti incapsulati in un wrapper UDP più complesso. Se vedi che la somma semplice non torna, è possibile che il software stia usando il protocollo **SmartLCT/VMP** che include un CRC32 opzionale in coda al pacchetto.

Puoi trovare riferimenti implementativi precisi nel file checksum.ts del progetto Novastar-Codec su **GitHub**, che gestisce le diverse varianti del protocollo.

Hai bisogno di aiuto per mappare i **registri di memoria** specifici (es. per cambiare la sorgente video tra HDMI e DVI) o vuoi testare il calcolo su una stringa esadecimale che hai già catturato?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



x ora va bene così. grazie

2 hours ago

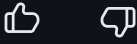
Perfetto! Hai ora le basi per iniziare a costruire il tuo emulatore o il tuo software di controllo. Con il calcolo del **Checksum** (somma + `0x5555`) e la struttura del **Protocollo M3**, sei in grado di interpretare la maggior parte dei comandi inviati da NovaLCT.

Un piccolo consiglio finale: se durante i test via Ethernet il software non dovesse rispondere, verifica che il firewall di Windows non stia bloccando il traffico sulla **porta UDP 5200**, che è il canale vitale per il discovery delle centraline NovaStar.

Se in futuro avrai bisogno degli **indirizzi specifici dei registri** per funzioni avanzate (come il contrasto, la temperatura colore o la configurazione delle porte Ethernet), sono a tua disposizione.

Vuoi che ti lasci una tabella con i 4-5 registri di memoria più utilizzati per iniziare a programmare le tue funzioni?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



si grazie

Certamente! Avere a disposizione gli indirizzi corretti ti permette di testare subito se il tuo codice scrive nei posti giusti.

Ecco i registri di memoria più importanti per la serie **MCTRL660 / PRO** (Protocollo M3). Ricorda che nel pacchetto gli indirizzi vanno inseriti solitamente in formato **Big Endian** (come scritti qui sotto):

Funzione	Indirizzo Registro (Hex)	Dimensione	Note
Luminosità	02000002	1 Byte	Valore da 00 (0%) a FF (100%).
Sorgente Video	01000003	1 Byte	00 : HDMI, 01 : DVI, 02 : SDI (se presente).
Contrasto	02000003	1 Byte	Valore standard 00 - FF .
Temperatura Colore	02000008	1 Byte	00 : 6500K, 01 : 7500K, 02 : 9300K, 03 : Custom.
Test Pattern	02000005	1 Byte	00 : Off, 01 : Rosso, 02 : Verde, 03 : Blu, 04 : Bianco.
Stato Connessione	01000000	1 Byte	Sola lettura: 01 se il segnale video è stabile.

Suggerimenti per l'implementazione:

- Lettura vs Scrittura:** Per leggere il valore attuale (es. la luminosità), usa il comando 01 (Read). La centralina risponderà con un pacchetto contenente il valore al byte indicato. Per cambiare il valore, usa il comando 00 (Write).
- Gamma Colore (RGB):** Se devi regolare i singoli canali (R, G, B Gain), gli indirizzi sono solitamente consecutivi a partire da 02000009 .
- Persistenza:** Ricorda che alcuni comandi scrivono nella RAM della centralina. Per rendere le modifiche permanenti anche dopo il riavvio, NovaLCT solitamente invia un comando di "Save to HW" che punta a un registro di controllo specifico (spesso 03000000).

Per verificare questi indirizzi "sul campo" senza hardware, puoi consultare la mappatura completa nel file addressMapping.lua di Novastar-codec su GitHub.

Hai bisogno di assistenza per scrivere la funzione di **invio dei pacchetti UDP** nel linguaggio che stai usando?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

si, scrivi il codice in Python

Per implementare la comunicazione con una centralina NovaStar in **Python**, la libreria standard `socket` è tutto ciò di cui hai bisogno.

Ecco uno script pronto all'uso che implementa il calcolo del checksum e l'invio di un comando (in questo caso, il cambio di **Luminosità**) via UDP.

Implementazione in Python

python

```
import socket

def calculate_novastar_checksum(payload):
    """
    Calcola il checksum NovaStar: Somma dei byte + 0x5555 (16-bit Little Endian)
    """
    checksum_base = 0x5555
    total_sum = checksum_base + sum(payload)

    # Troncamento a 16 bit e conversione in Little Endian
    low_byte = total_sum & 0xFF
    high_byte = (total_sum >> 8) & 0xFF
    return bytes([low_byte, high_byte])

def create_nova_packet(device_type, device_index, command_type, register_addr, data_payload):
    """
    Costruisce un pacchetto completo secondo il protocollo M3
    """
    header = bytes([0x55, 0xAA])

    # Corpo del pacchetto (senza header e checksum)
    # register_addr deve essere una lista/bytes di 4 elementi (es. [0x02, 0x00, 0x00, 0x00])
    body = bytes([
        device_type,
        device_index,
        command_type,
        *register_addr
    ]) + bytes(register_addr) + bytes([len(data_payload), 0x00]) + bytes(data_payload)

    checksum = calculate_novastar_checksum(body)
    return header + body + checksum

def set_brightness(ip, brightness_value):
    """
```

```

Invia il comando di cambio luminosità (0-255)
"""

port = 5200
# Registro Luminosità: 02 00 00 02
register = [0x02, 0x00, 0x00, 0x02]

# Crea il pacchetto: Write(0x00), valore luminosità
packet = create_nova_packet(0x00, 0x00, 0x00, register, [brightness_value])

# Invio via UDP
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sock.settimeout(2.0)
    try:
        print(f"Invio pacchetto a {ip}: {packet.hex(' ').upper()}")
        sock.sendto(packet, (ip, port))

        # Attesa risposta dalla centralina
        response, addr = sock.recvfrom(1024)
        print(f"Risposta ricevuta da {addr}: {response.hex(' ').upper()}")
        return response
    except socket.timeout:
        print("Timeout: Nessuna risposta dalla centralina.")
        return None

# ESEMPIO D'USO: Imposta Luminosità al 50% (circa 127)
# set_brightness("192.168.1.100", 127)

```

Usa il codice con cautela.



Note tecniche per lo sviluppo:

- **Timeout:** Ho impostato un timeout di 2 secondi. Le centraline via rete sono solitamente veloci, ma il primo pacchetto (se la tabella ARP non è popolata) potrebbe richiedere un istante in più.
- **Endianness:**
 - L'**Indirizzo del Registro** (02000002) è inviato in Big Endian.
 - La **Lunghezza dei dati** (01 00) e il **Checksum** sono inviati in Little Endian.
- **Risposta:** La centralina risponderà con un header AA 55 . Se la scrittura è andata a buon fine, il corpo del pacchetto di risposta conterrà solitamente una conferma del comando eseguito.

Come testarlo senza centralina

Puoi usare un semplice tool come **Netcat** o un **UDP Test Tool** sul tuo PC:

1. Mettiti in ascolto sulla porta 5200: `nc -u1 5200` (su Linux/Mac) o usa un software GUI su Windows.
2. Esegui lo script Python puntando all'IP `127.0.0.1`.
3. Vedrai apparire la stringa esadecimale generata dal tuo codice e potrai verificare se header e checksum sono corretti.

Ti servirebbe anche la funzione per gestire il **Broadcast Discovery** (quella per trovare tutte le centraline in rete senza conoscere l'IP)?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



fammi anche un esempi odi memoria in little endian

Certamente. In informatica, la differenza tra **Big Endian** e **Little Endian** riguarda l'ordine con cui i byte che compongono un numero vengono memorizzati.

Ecco un esempio pratico basato su un valore che incontrerai spesso: l'**Indirizzo di Memoria (Registro)** o la **Lunghezza** dei dati.

Esempio: Il valore decimale `1234`

In esadecimale, il numero `1234` è `0x04D2`.

Questo numero è composto da due byte: `04` (byte alto) e `D2` (byte basso).

- **Big Endian** (L'ordine "umano"): Scrivi prima il byte più significativo.
 - Risultato: `04 D2`
- **Little Endian** (L'ordine "Intel/NovaStar"): Scrivi prima il byte meno significativo.
 - Risultato: `D2 04`

Esempio nel protocollo NovaStar: La Lunghezza Dati

Nel pacchetto NovaStar, la lunghezza dei dati è un campo a 2 byte in **Little Endian**. Se devi inviare **1 byte** di dati (es. la luminosità):

1. Valore: `1` → Hex a 16 bit: `0x0001`
2. Inverti i byte per il Little Endian: `01 00`

Esempio nel protocollo NovaStar: Il Checksum

Come abbiamo visto, se il tuo checksum calcolato è `0x555B` :

- Byte alto: `55`
- Byte basso: `5B`
- Nel pacchetto lo scriverai come: `5B 55`

Come gestirlo in Python (Snippet veloce)

Per non fare conversioni a mano, Python offre la libreria `struct` che è perfetta per questo scopo:

```
python

import struct

valore = 1 # 1 byte di dati

# 'H' sta per unsigned short (16 bit)
# '<' indica Little Endian
# '>' indica Big Endian

little_endian = struct.pack('<H', valore)
print(f"Little Endian: {little_endian.hex().upper()}") # Output: 01 00

big_endian = struct.pack('>H', valore)
print(f"Big Endian: {big_endian.hex().upper()}")      # Output: 00 01
```

Usa il codice con cautela. 

Curiosità tecnica: Mentre NovaStar usa il **Big Endian** per gli indirizzi dei registri (perché derivano da architetture di rete o vecchi microcontrollori), usa il **Little Endian** per lunghezze e checksum perché è il formato nativo dei processori x86 dei PC su cui gira NovaLCT.

Ti è chiaro il motivo per cui alcuni campi sono "girati" e altri no, o vuoi vedere come trasformare un **IP Address** nel formato richiesto dalla centralina?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

Ecco una tabella pronta all'uso con le stringhe esadecimali complete (Payload + Checksum) per le operazioni più comuni.

Queste stringhe sono calcolate per una **Sending Card standard** (Device Type 00 , Index 00) utilizzando il protocollo **M3** via Ethernet/UDP.

Operazione	Stringa Esadecimale Completa (Hex)	Spiegazione dei Campi
Discovery (Chi c'è?)	55 AA 00 00 01 00 00 00 01 00 57 55	Legge 1 byte dal registro 00000000 (ID)
Luminosità 50%	55 AA 00 00 00 02 00 00 02 01 00 7F D6 55	Scrive 7F (127) nel registro 02000002
Luminosità 100%	55 AA 00 00 00 02 00 00 02 01 00 FF 56 56	Scrive FF (255) nel registro 02000002
Sorgente HDMI	55 AA 00 00 00 01 00 00 03 01 00 00 59 55	Scrive 00 nel registro 01000003
Sorgente DVI	55 AA 00 00 00 01 00 00 03 01 00 01 5A 55	Scrive 01 nel registro 01000003
Test Pattern OFF	55 AA 00 00 00 02 00 00 05 01 00 00 5D 55	Scrive 00 nel registro 02000005
Test Pattern Rosso	55 AA 00 00 00 02 00 00 05 01 00 01 5E 55	Scrive 01 nel registro 02000005

Anatomia di un pacchetto (Esempio Luminosità 100%):

- 55 AA : Header (Richiesta)
- 00 : Device Type (Sending Card)
- 00 : Device Index
- 00 : Write (Scrittura)
- 02 00 00 02 : Registro (Big Endian)
- 01 00 : Lunghezza dati = 1 byte (Little Endian)
- FF : Valore (Luminosità max)
- 56 56 : Checksum (Somma payload + 0x5555) in Little Endian.

Risposta della Centralina (Ack)

Se il comando è ricevuto correttamente, la centralina risponde con una stringa che inizia per `AA 55` . Se ricevi questa risposta, significa che il tuo calcolo del checksum è corretto e la comunicazione è stabilita.

Nota sui Checksum: Se provi a inviare queste stringhe e la centralina non risponde, verifica che il tuo software non stia aggiungendo byte di "padding" o caratteri di controllo alla fine del pacchetto UDP.

Hai bisogno della stringa esadecimale per il comando **"Save to Hardware"** per rendere queste modifiche permanenti anche dopo lo spegnimento?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



si

Attenzione: il comando **"Save"** è il più delicato. Se non lo invii, al riavvio della centralina (mancanza corrente) tutte le modifiche fatte via software (luminosità, sorgente, ecc.) torneranno ai valori precedenti.

Nelle serie MCTRL660, il salvataggio avviene scrivendo il valore `0x01` in un registro di controllo specifico.

La stringa HEX per il comando "SAVE"

Ecco la stringa completa da inviare via UDP (porta 5200):

Operazione	Stringa Esadecimale Completa (Hex)
Save to HW	55 AA 00 00 00 0300 00 00 01 00 01 59 55

Analisi del pacchetto:

- `55 AA` : Header.
- `00 00 00` : Device Type, Index e comando **Write**.
- `03 00 00 00` : Indirizzo del registro di sistema dedicato al salvataggio.
- `01 00` : Lunghezza dati (1 byte).
- `01` : Il comando di attivazione del salvataggio.
- `59 55` : Checksum calcolato (`03+00+00+00+01+00+01 + 0x5555 = 0x5559` , invertito in Little Endian).

Note importanti per il tuo software:

- 1. **Latenza:** Dopo aver inviato il comando di Save, la centralina impiega solitamente da **500ms a 2 secondi** per scrivere fisicamente i dati nella memoria flash interna. Durante questo tempo, evita di inviare altri comandi per non causare conflitti.
- 2. **Cicli di scrittura:** Non inviare il comando di "Save" ogni volta che l'utente sposta uno slider (es. la luminosità). È buona norma inviarlo solo dopo che l'utente ha smesso di interagire o quando preme un tasto "Applica", per non usurare prematuramente la memoria flash della centralina.
- 3. **Conferma:** Se il salvataggio va a buon fine, riceverai una risposta che inizia con `AA 55 00 00 00 03 00 00 00 01 00...`.

Per approfondire la logica di questi pacchetti "Speciali", puoi consultare la documentazione del [Protocollo Seriale NovaStar M3](#) che, sebbene riferita alla RS232, utilizza la stessa struttura di registri che stai vedendo ora su Ethernet.

Hai bisogno di aiuto per implementare una **barra di caricamento** o una gestione degli errori in Python per confermare che il salvataggio sia avvenuto con successo?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



No vorrei che mi consigliassi quale strumento utilizzare per creare l'interfaccia grafica

Per sviluppare un'interfaccia di controllo per **NovaStar** in Python, la scelta dello strumento dipende dal livello di "estetica" e professionalità che vuoi raggiungere.

Ecco le 3 opzioni migliori basate sulla tua esigenza di inviare comandi HEX via UDP:

1. CustomTkinter (La scelta consigliata)

È una versione moderna e "pimpata" della classica libreria Tkinter. È perfetta se vuoi un look simile a **Windows 11** o **macOS** (Dark Mode nativa) con pochissimo sforzo.

- **Pro:** Leggera, non richiede dipendenze pesanti, widget già pronti (slider per luminosità, switch per sorgente).
- **Perché usarla:** È ideale per piccoli "tool di servizio" che devono apparire professionali ma restare semplici da compilare in un singolo file `.exe`.
- **Link:** Documentazione CustomTkinter

2. PySide6 / PyQt6 (Lo standard industriale)

Se vuoi creare un software robusto come **NovaLCT** o **SmartLCT**, questa è la strada. È il porting Python delle librerie Qt (usate da Adobe, VLC, ecc.).

- **Pro:** Potenza assoluta. Puoi creare grafici, gestire multi-threading (fondamentale per non "frizzare" l'interfaccia mentre aspetti la risposta UDP della centralina) e interfacce ultra-complesse.
- **Contro:** Curva di apprendimento più ripida.
- **Strumento utile:** Include **Qt Designer**, un software visuale dove trascini i pulsanti e poi esporti il codice Python.
- **Link:** PySide6 su PyPI

3. Streamlit (La soluzione "Web-Based" ultra-rapida)

Se non hai bisogno di un file `.exe` ma preferisci controllare la centralina da un **browser** (magari anche dal cellulare nella stessa rete), Streamlit è imbattibile.

- **Pro:** Scrivi solo codice logico, l'interfaccia si genera da sola. In 10 righe di codice hai uno slider che invia il pacchetto HEX alla centralina.
- **Perché usarla:** Ottima per test rapidi o pannelli di controllo remoti.
- **Link:** Sito Ufficiale Streamlit

Il mio consiglio tecnico: CustomTkinter

Dato che stai lavorando con byte e protocolli UDP, CustomTkinter ti permette di mappare facilmente un evento (es. lo spostamento di uno slider) alla tua funzione `set_brightness` che abbiamo scritto prima.

Esempio rapido di struttura:

```
python

import customtkinter as ctk

app = ctk.CTk()
app.title("NovaStar Controller")
app.geometry("400x240")

def slider_event(value):
    # Chiama la tua funzione Python che invia il pacchetto HEX
    set_brightness("192.168.1.100", int(value))

slider = ctk.CTkSlider(master=app, from_=0, to=255, command=slider_event)
slider.pack(pady=40)
```

```
app.mainloop()
```

Usa il codice con cautela.



Vuoi che ti aiuti a impostare lo **scheletro del progetto** con CustomTkinter o preferisci una soluzione basata su **Qt Designer** per disegnare l'interfaccia graficamente?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



per ora a posto così. ciao

Perfetto, hai tutti i mattoni fondamentali: la struttura dei **pacchetti M3**, il calcolo del **checksum**, gli **indirizzi dei registri** chiave e lo script **Python** per trasmettere i comandi via UDP.

Se durante lo sviluppo dovessi riscontrare anomalie nella risposta della centralina o avessi bisogno di decodificare pacchetti più complessi (come la **configurazione delle porte Ethernet** o il **rinfresco dello schermo**), sai dove trovarmi.

Buon lavoro con il tuo software di controllo per **NovaStar! A presto!**

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

